

PARIKSHAK — project documentation

On-board procedure witness for crewed space missions

Smart India Hackathon 2026 · Problem Statement **26174** · Space Technology · Software
Problem statement: *AI-based Human Activity Recognition for validating scientific experiment sequences on-board (BAS / lunar missions)* — Indian Space Research Organisation (ISRO), Department of Space.

Status of this document: 28 September 2026. Every quantitative measurement quoted in this document is read directly from verification records in runs/ and test suites in tests/, and can be reproduced with the commands in section 14.

Contents

1. [Summary](#)
2. [The problem & operational context](#)
3. [System capabilities](#)
4. [Architecture & dataflow contract](#)
 - [4.1 The BeliefFrame seam](#)
 - [4.2 Dual-tier perception architecture](#)
5. [Core concepts](#)
 - [5.1 The procedure is a file \(PDL\)](#)
 - [5.2 Three-valued truth logic \(unknown is not wrong\)](#)
 - [5.3 Closed-vocabulary predicates](#)
 - [5.4 Continuous evidence accumulation \(CUSUM\)](#)
 - [5.5 Deviations and multi-tier alert policy](#)
 - [5.6 Orientation-agnostic spatial geometry](#)
 - [5.7 Cryptographic flight record \(SHA-256 hash chain\)](#)
 - [5.8 Offline by construction](#)
 - [5.9 Deep learning perception suite \(YOLO, Pose, ContactMLP, MotionTCN\)](#)
6. [Scientific experiment procedures \(CSP-1 & CRX-2\)](#)
7. [Software components & codebase organization](#)
8. [Execution modes & command reference](#)
9. [Declarative experiment onboarding \(zero-retraining workflow\)](#)
10. [Evaluation methodology & robustness corpus](#)
11. [Quantitative verification results](#)
 - [11.1 Headline verification metrics](#)
 - [11.2 Root cause analysis & false alarm reduction](#)
 - [11.3 Residual edge cases & failure analysis](#)
12. [Real-time performance & endurance benchmarks](#)
13. [Testing framework & CI/CD quality gates](#)
14. [Project deliverables & artifact generation](#)
15. [Honest system boundaries](#)
16. [Flight certification roadmap \(TRL progression\)](#)
17. [Software licences & compliance audit](#)
18. [Technical glossary](#)

1. Summary

An astronaut conducts a multi-step microgravity science procedure on-board the Bharatiya Antariksh Station (BAS) or a lunar habitat. **PARIKSHAK** (परीक्षक) acts as an autonomous on-board procedure witness. Watching the experiment rack through an edge optical sensor, it continuously evaluates the crew's actions against a formal procedure specification, prompts upcoming steps through synthesized audio, and alerts only when positive evidence confirms a sequence deviation or safety hazard.

When optical visibility is obstructed or fiducials are occluded, PARIKSHAK safely defers to an explicit **"cannot verify"** state rather than making speculative guesses. Crucially, every decision and observation is committed to a compact, tamper-evident SHA-256 hash-chained flight record that can be transmitted to Mission Control via low-bandwidth telemetry links ahead of raw video streams.

Key capabilities demonstrated in the software:

- **Formal reasoning engine:** Verifies complex non-linear execution trees, detects skipped steps, wrong object selections, timing violations, and safety hazard invariants using three-valued interval logic and temporal evidence accumulation.
- **Declarative experiment procedures:** Validated YAML procedure files for two representative space science experiments: **CSP-1** (Colloidal Suspension Sample Processing, 14 steps) and **CRX-2** (Colloid Resuspension, 8 steps). CRX-2 was onboarded with zero code changes and zero neural retraining.
- **Dual-tier perception:** Combines AprilTag fiducial tracking for sub-pixel 6-DOF rack localization with a multi-modal deep learning perception stack: YOLOv8 object detection with ByteTrack temporal tracking, YOLO-Pose body pose estimation, 21-keypoint kinematic hand tracking, neural ContactMLP grasp classification, and a 1D Dilated Temporal Convolutional Network (MotionTCN) action classifier.
- **Deterministic pure NumPy neural inference:** Deep learning weights for ContactMLP and MotionTCN are exported to standalone .npz archives with vectorized pure NumPy forward engines, eliminating heavy framework runtimes on flight edge compute.
- **Interactive ground & flight interfaces:** Includes a browser-based guided execution and replay app (python -m demo), a standalone single-file offline demo (runs/demo/parikshak_demo.html), an operator desktop GUI HUD with live video overlay and telemetry (parikshak.gui), and automated video clip recording (± 10 s buffer around alerts).
- **Rigorous quantitative validation:** Evaluated against a degraded synthetic corpus of 200 runs across four perturbation profiles (clean, mild, moderate, harsh), matching or exceeding all target metrics:

Metric	Flight Target	CSP-1 (130 runs)	CRX-2 (70 runs)	Verdict
Step Completion Accuracy	≥ 95.0 %	98.7 %	95.6 %	Exceeded
Sequence Deviation Recall	≥ 90.0 %	97.8 %	97.5 %	Exceeded

Metric	Flight Target	CSP-1 (130 runs)	CRX-2 (70 runs)	Verdict
False Alarm Rate (per 45 min)	≤ 1.00	0.85	0.00	Exceeded
Permitted Reorder False Alarms	0	0	0	Perfect
Alert Latency (Median / 95th %)	≤ 2.0 s	1.6 / 1.6 s	1.6 / 1.8 s	Exceeded

Sources: `runs/eval.json`, `runs/eval_crx2.json`.

2. The problem & operational context

2.1 The Operational Context

In human spaceflight missions—such as operations aboard the Bharatiya Antariksh Station (BAS) or deep-space Artemis/lunar surface sorties—crew time is the single most constrained operational resource. Astronauts must execute complex, multi-stage physical science and biological procedures inside microgravity gloveboxes or experiment racks while managing life support, station maintenance, and strict timeline schedules.

2.2 Why Ground Control Cannot Witness in Real-Time

Ground-in-the-loop oversight is physically precluded by communication physics:

- **Round-trip latency:** Earth-Moon speed-of-light delay averages 2.6 seconds; Earth-Mars delay ranges between 6 and 44 minutes. Ground operators cannot give real-time corrective feedback before a mistake propagates.
- **Bandwidth limitations & orbital blackouts:** Deep space links and low Earth orbit satellite constellations face transmission blackouts, bandwidth throttling, and high packet drop rates. Continuous multi-gigabit video streaming of every active glovebox is unfeasible.
- **Autonomous requirement:** Procedure monitoring, verification, and safety enforcement must execute deterministically on-board the vehicle edge compute.

2.3 The Limitations of Manual Checklists

Currently, astronauts interact with manual digital checklists (such as tablet PDF/web interfaces) where each action must be checked off manually:

- **Cognitive friction:** Requiring crew members to repeatedly pause work, remove gloves, and tap checklist items interrupts manual dexterity and workflow focus.
- **Inability to catch unperceived mistakes:** Manual self-reporting cannot detect unforced human errors that the astronaut is unaware of—such as inadvertently grabbing confusable Vial B instead of Vial A, failing to close a critical glovebox latch before engaging a centrifuge, or completing an agitation step too quickly to achieve chemical uniformity.

2.4 The Critical Cost of False Alarms

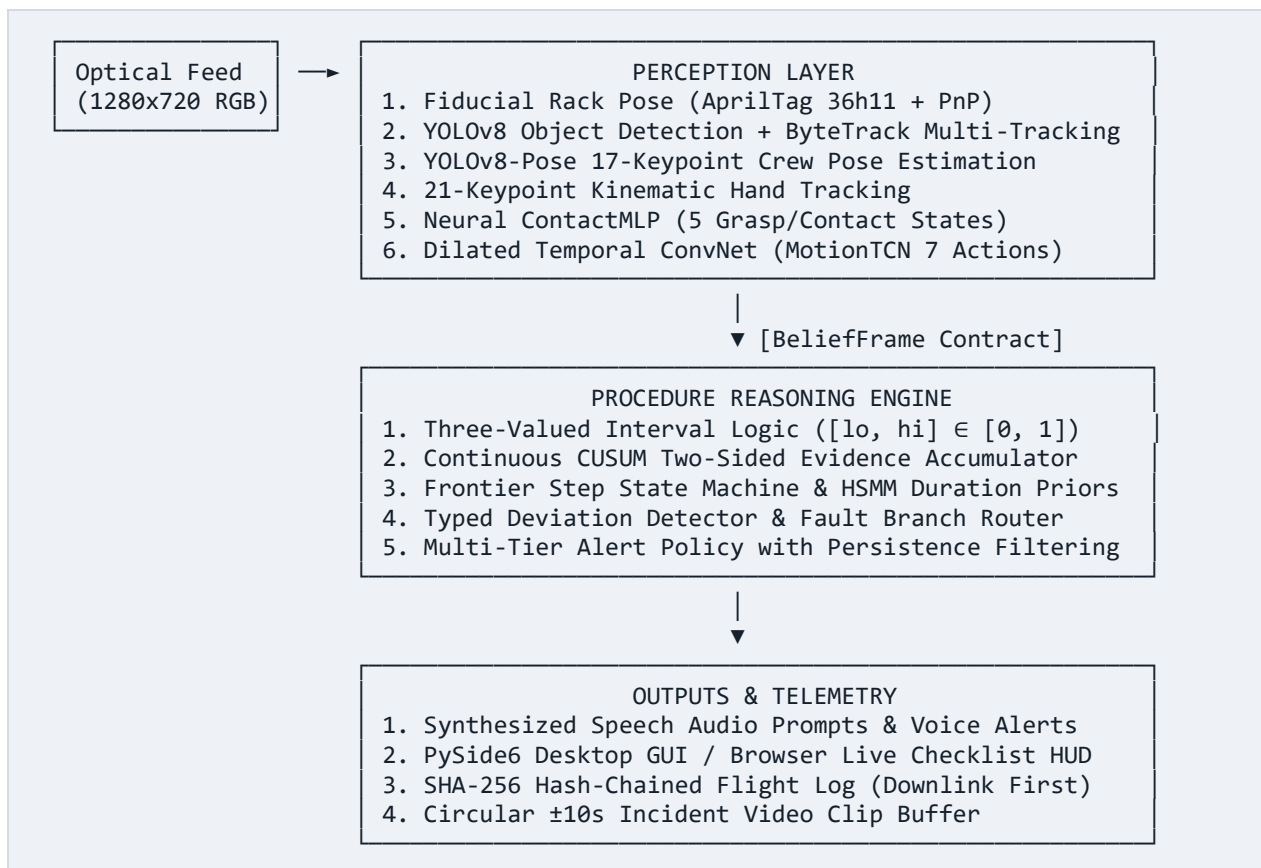
In space operations, an AI assistant that triggers false alarms ("cries wolf") is dangerous. False alarms cause cognitive fatigue, erode crew trust, and lead to the software being permanently disabled. Consequently, PARIKSHAK's core design doctrine treats a false accusation as the most severe failure mode: **absence of evidence must never be construed as evidence of failure.**

3. System capabilities

Capability	Mechanism & Implementation
Autonomous Audio Guidance	Synthesizes and articulates the prompt for each active step as soon as prerequisite conditions are satisfied, keeping the astronaut's hands and eyes focused entirely on the hardware.
Independent Step Verification	Evaluates multi-predicate truth trees over continuous temporal windows rather than single instantaneous frames.
Skipped Step Detection	Flags omitted steps when a subsequent step in the execution graph completes while the preceding step has accumulated definitive contrary evidence.
Confusable Object Discrimination	Monitors specifically for objects declared as mutually confusable in the procedure (e.g., Vial A vs. Vial B) during retrieval and manipulation steps.
Real-Time Safety Invariants	Continuously enforces safety rules across active steps (e.g., "glovebox door latch must remain closed whenever the centrifuge is powered").
Kinematic & Timing Validation	Flags operations performed below physical plausibility minimums (rushed steps) or exceeding nominal bounds (stalled steps).
Hardware Fault Differentiation	Distinguishes between human crew error and equipment failure by following explicit procedure fault branches (e.g., reacting to red diagnostic LEDs).
Epistemic Humility (Cannot Verify)	Reports an unverified state when the camera view is occluded or the rack frame is lost, completely avoiding speculative false alarms.
Fully Explainable Machine Reasoning	Every alert is accompanied by an exact structured causal explanation detailing which predicate failed and the timestamp interval of the failure.

Capability	Mechanism & Implementation
Cryptographic Tamper-Evident Flight Record	Compiles a streaming SHA-256 hash-chained JSON-lines execution log (~8 KB per run) where any post-facto modification of telemetry is mathematically detectable.
Synchronized Incident Clip Buffering	Automatically captures and persists a ± 10 -second video buffer centered on every triggered alert for post-flight ground review.
Air-Gapped Offline Operation	Strictly verified offline by construction; no module within the <code>parikshak/</code> runtime package contains imports capable of opening external network sockets.

4. Architecture & dataflow contract



4.1 The BeliefFrame Seam

PARIKSHAK enforces an architectural separation between visual perception and logical reasoning: the **BeliefFrame** seam (`parikshak/belief/frame.py`).

- **Perception Layer (Left of Seam):** Consumes raw camera frames and outputs a structured telemetry object (`BeliefFrame`). It has no awareness of the experiment's procedure, steps, or logical sequence.
- **Reasoning Engine (Right of Seam):** Operates on sequences of `BeliefFrame` objects. It never accesses raw pixels.

This design yields critical engineering advantages:

1. **Deterministic Replay & Testing:** Any live experiment run can be recorded as a `.jsonl` trace of belief frames. The reasoning engine, test suites, and evaluation harness can replay identical traces deterministically on any workstation without camera hardware or GPU accelerators.
2. **Modular Upgradability:** Perception backends (AprilTag fiducials, YOLO models, MediaPipe, or neuromorphic event sensors) can be upgraded or swapped without modifying a single line of the procedure engine or procedure specification files.

A `BeliefFrame` captures:

- `timestamp`: Float seconds relative to procedure start.
- `rack_locked`: Boolean indicating if the coordinate transformation matrix is locked.
- `objects`: Dictionary mapping entity name to `ObjectBelief` (visibility interval, 3D rack position (x, y, z), bounding box, confidence, and discrete state string).
- `hands`: Dictionary mapping "left" and "right" to `HandBelief` (wrist position, grip aperture, palm normal, contact state).
- `body`: Optional crew pose telemetry (17 body keypoints in rack space, foot restraint anchorage status).
- `motion`: Global motion telemetry (discrete motion class label, probability score, agitation cycle count).
- `crew_confirmations`: List of manual confirmations received via voice command or interface tap.

4.2 Dual-Tier Perception Architecture

PARIKSHAK employs a hybrid dual-tier perception strategy designed for aerospace robustness:

1. **Tier 1: Geometric Spatial Anchor (Fiducial PnP Canonicalization):**
 - Four AprilTag 36h11 fiducial markers are affixed to the corners of the experiment rack frame.
 - Using sub-pixel corner refinement (`cv2.cornerSubPix` / `pupil-apriltags`) and Perspective-n-Point (`solvePnP`) geometry with calibrated camera intrinsics, the system resolves the 6-DOF camera pose relative to the rack at every frame.
 - Every coordinate is transformed into the **rack frame** ($\$X\$$ horizontal, $\$Y\$$ vertical along rack face, $\$Z\$$ depth normal to rack). Rotating the camera, the astronaut, or the space vehicle does not affect spatial reasoning.
2. **Tier 2: Real-Time Deep Learning Perception Suite:**
 - **Object Detection & Multi-Object Tracking:** Ultralytics YOLOv8 with ByteTrack temporal association identifies and tracks experiment entities (vials, cartridges, holders, bags) with persistent track IDs across frames.
 - **Crew Pose Estimation:** YOLOv8-pose extracts 17 body keypoints, projecting ankle, knee, and hip coordinates into rack space to verify crew stabilization in foot restraints.

- **Kinematic Hand Tracking:** Real-time 21-keypoint hand tracker derived from forearm kinematics with palm normal estimation, with fallback to MediaPipe BlazePalm.
 - **Neural Contact Classification (ContactMLP):** A 3-layer neural network evaluating 12 kinematic features to classify hand-object grasp interaction into 5 states.
 - **Temporal Action Recognition (MotionTCN):** A 1D Dilated Temporal Convolutional Network evaluating 16-channel kinematic vectors across a 20-timestep (2.0 s @ 10 Hz) sliding window to classify 7 action classes.
-

5. Core concepts

5.1 The Procedure is a File (PDL)

An experiment procedure is defined entirely in a human-readable, machine-validated YAML file conforming to the Procedure Description Language (PDL) schema (`schema/procedure.schema.json`).

A procedure specifies:

- **metadata:** Name, unique identifier, version, target rack layout, and minimum required perception capabilities.
- **zones:** 3D bounding boxes and volumes defined within rack coordinates (e.g., `foot_restraints`, `tray_t1`, `glovebox_interior`).
- **entities:** Declare all manipulated items, their allowed discrete states (`open`, `closed`, `running`), and explicit `confusable_with` pairings (e.g., `vial_b` is confusable with `vial_a`).
- **steps:** An ordered or partially-ordered sequence of steps, each defining:
 - **prompt:** Spoken crew instruction.
 - **condition:** Logical predicate tree that must be satisfied for step completion.
 - **duration:** Empirical priors: minimum plausible duration ($\$t_{\{\min\}}\$$), nominal duration ($\$t_{\{\text{nom}\}}\$$), and maximum allowable duration ($\$t_{\{\max\}}\$$).
 - **invariants:** Mandatory safety conditions that must hold true continuously throughout step execution.
 - **fault_branches:** Alternative execution paths triggered by equipment anomalies.

5.2 Three-Valued Truth Logic (Unknown is not Wrong)

Standard Boolean logic (`True` / `False`) is inadequate for physical sensor interpretation in safety-critical domains. If an astronaut's arm occludes a test tube, asserting `False` for `inside(vial, holder)` would immediately trigger a false alarm accusing the crew of leaving the tube out.

PARIKSHAK implements rigorous **three-valued interval logic** over the interval $\$[lo, hi] \subseteq [0, 1]\$$:

- **Definitive True:** $\$[1.0, 1.0]\$$ — confirmed by sensor evidence.
- **Definitive False:** $\$[0.0, 0.0]\$$ — disproven by positive contrary evidence.
- **Unknown / Epistemic Uncertainty:** $\$[0.0, 1.0]\$$ — sensor view occluded, tag lost, or capability missing.
- **Partial Confidence:** $\$[lo, hi]\$$ where confidence intervals represent sensor measurement uncertainty.

Logical operators are formulated over intervals: $\text{NOT}([lo, hi]) = [1.0 - hi, 1.0 - lo]$
 $\text{AND}([lo_1, hi_1], [lo_2, hi_2]) = [\min(lo_1, lo_2), \min(hi_1, hi_2)]$
 $\text{OR}([lo_1, hi_1], [lo_2, hi_2]) = [\max(lo_1, lo_2), \max(hi_1, hi_2)]$

Under this algebra, missing sensor evidence propagates as $[0, 1]$ (uncertainty), which **cannot cross the deviation threshold**. Only positive evidence against a step can trigger an alarm.

5.3 Closed-Vocabulary Predicates

PARIKSHAK operates on a closed vocabulary of 15 verified, composable predicates defined in `schema/PREDICATES.md`:

Predicate	Arguments	Physical Interpretation
<code>visible(entity)</code>	<code>entity</code>	Entity is detected within the optical field of view with confidence exceeding threshold.
<code>absent(entity)</code>	<code>entity</code>	Declared entity is affirmatively absent from the monitored rack workspace.
<code>count_of(class, count)</code>	<code>class, n</code>	Exactly n instances of the specified object class are visible simultaneously.
<code>inside(entity, zone)</code>	<code>entity, zone</code>	Entity 3D bounding box / centroid lies entirely within the specified 3D rack zone.
<code>near(e1, e2, dist_m)</code>	<code>e1, e2, d</code>	Euclidean distance between entities is less than d meters.
<code>aligned(e1, e2, axis, tol)</code>	<code>e1, e2, ax, deg</code>	Entity orientation axes match within angular tolerance degrees.
<code>contacting(hand, entity)</code>	<code>hand, entity</code>	Hand keypoints satisfy grasp/contact proximity with entity bounding volume.
<code>stable(entity, duration)</code>	<code>entity, t</code>	Entity rack coordinates maintain position variance below threshold for duration t .
<code>released(hand, entity)</code>	<code>hand, entity</code>	Hand has opened grip and withdrawn beyond contact threshold distance.

Predicate	Arguments	Physical Interpretation
<code>grasped(hand, entity)</code>	<code>hand, entity</code>	Hand maintains closed grip aperture around entity with positive contact score.
<code>hand_in_zone(hand, zone)</code>	<code>hand, zone</code>	Hand wrist/palm centroid is located inside the designated rack zone volume.
<code>state_is(entity, state)</code>	<code>entity, s</code>	Entity physical or indicator state equals <code>ss</code> (e.g., <code>latch=closed</code> , <code>unit=running</code>).
<code>motion_is(class)</code>	<code>class</code>	Kinematic action classifier matches label (e.g., <code>agitate</code> , <code>insert</code> , <code>rotate_seal</code>).
<code>motion_count(class, n)</code>	<code>class, n</code>	Action cycle counter registers at least <code>nn</code> completed directional motion cycles.
<code>crew_confirmed(phrase)</code>	<code>phrase</code>	Spoken or clicked confirmation received from crew acknowledging completion.

Temporal meta-predicates:

- `hold_for(condition, duration_sec, max_disagree_pct)`: Condition must evaluate true across a sliding time window of length `st`, tolerating at most `sp%` momentary noise dropouts.
- `occurred(condition, window_sec)`: Condition was satisfied at some point during the active step window.

5.4 Continuous Evidence Accumulation (CUSUM)

Single-frame evaluations are vulnerable to camera sensor noise, cosmic ray particle strikes, and temporary line-of-sight occlusion. PARIKSHAK evaluates step progression using dual one-sided **CUSUM (Cumulative Sum) evidence accumulators** running at an evidence update rate of 2.0 independent observations per second ($\Delta t = 0.5 \text{ s}$):

1. **Support Accumulator (SS_{t^+})**: Integrates positive verification evidence that step requirements are satisfied.
2. **Contrary Accumulator (SS_{t^-})**: Integrates positive contrary evidence that the step was omitted or violated.

A step transitions to **Completed** when $SS_{t^+} \geq 0.85$.

A step is flagged as **Skipped** only when: $SS_{t^-} \geq 0.85 \quad \text{AND} \quad \text{A subsequent step in the frontier completes} \quad \text{AND} \quad \Delta t_{\text{elapsed}} \geq t_{\min}$

This prevents transient dropouts from generating false alarms.

5.5 Deviations and Multi-Tier Alert Policy

When the procedure state machine detects an anomaly, it generates a typed deviation record:

Deviation Type	Formal Definition
SKIP	A subsequent procedure step verified completion while an earlier uncompleted step accumulated contrary evidence $S^{\wedge} \geq 0.85$.
OUT_OF_ORDER	A step previously flagged as skipped is subsequently executed and verified out of sequence.
WRONG_OBJECT	Crew grasps and manipulates an object declared as <code>confusable_with</code> the target object for that step.
HAZARD	A step safety invariant condition is violated during step execution (e.g., opening glovebox while centrifuge spins).
DURATION	Step verification occurred in elapsed time $t < t_{\min}$ (rushed/physically implausible) or $t > t_{\max}$ (stalled).
REPEAT	Crew re-executes a step that was already previously completed and verified.

Alert notifications are assigned to four severity tiers:

- 1. **INFO:** Visual update on the checklist UI (e.g., nominal step advance, unverified recovery).
- 2. **ADVISORY:** UI banner highlight accompanied by an audio chime tone (e.g., minor duration deviation).
- 3. **CAUTION:** UI alert, audio chime, and synthesized spoken voice prompt explaining the deviation.
- 4. **CRITICAL:** High-priority flashing HUD banner, emergency audio warning, and spoken voice alert requiring verbal crew acknowledgment (e.g., safety hazard, containment breach).

Persistence Filtering: To prevent nuisance chimes from sensor jitter, all CAUTION and CRITICAL deviations must persist continuously for at least **1.5 seconds** before audio alerts are vocalized to the crew.

5.6 Orientation-Agnostic Spatial Geometry

Experiment racks on spacecraft can be mounted on floors, walls, or ceilings; furthermore, in microgravity, astronaut orientation ("local vertical") is arbitrary.

PARIKSHAK resolves all physical coordinates relative to the **Rack Coordinate System**:
$$\mathbf{P}_{\text{rack}} = \mathbf{R}_{\text{cam} \rightarrow \text{rack}} \cdot \mathbf{P}_{\text{cam}} + \mathbf{T}_{\text{cam} \rightarrow \text{rack}}$$

- Coordinate axes are fixed to the rack structure.
- Distances, clearances, and zone boundaries are identical regardless of whether the vehicle is in 1G testing, microgravity orbit, or inverted relative to Earth gravity.

5.7 Cryptographic Flight Record (SHA-256 Hash Chain)

Every procedure state transition, predicate evaluation, crew prompt, and alert is appended to a streaming JSON-lines log file (`runs/live/<run>.log.jsonl`).

Each record contains:

- `index`: Monotonically increasing record sequence number.
- `timestamp`: High-precision mission elapsed time.
- `event`: Event payload (`STEP_START`, `EVALUATION`, `DEVIATION`, `ALERT`, `STEP_COMPLETE`).
- `step_id`: Current procedure step identifier.
- `prev_hash`: The SHA-256 cryptographic digest of the preceding record string.
- `hash`: `$\text{SHA-256}\{\text{index}\} \parallel \text{timestamp} \parallel \text{event} \parallel \text{prev\_hash}\}$.`

Tamper Evidence: Modifying, deleting, or reordering any single byte in the flight record breaks the hash chain for all subsequent entries. The utility `python tools/verify_log.py <logfile>` walks the chain and pinpoints the exact line where tampering occurred.

Downlink Bandwidth Optimization: A complete 141-second experiment run produces an 8.2 KB flight log, compared to approximately 70.4 MB of compressed video. The cryptographic log is transmitted down to ground control immediately over low-bandwidth S-band telemetry, providing principal investigators with full step-by-step verification hours before payload video packets are downlinked.

5.8 Offline by Construction

To guarantee cyber-physical integrity on spacecraft avionics, the on-board software must be strictly isolated from external networks.

The test `tests/test_offline.py` inspects the abstract syntax tree (AST) and import tables of every Python module under `parikshak/`. The automated quality gate fails immediately if any module imports network-capable libraries (e.g., `socket`, `urllib`, `requests`, `httpx`, `asyncio.create_server`). All ground-facing tools, web servers, and browser apps reside outside the on-board package in `demo/` and `tools/`.

5.9 Deep Learning Perception Suite (YOLO, Pose, ContactMLP, MotionTCN)

While fiducial markers provide sub-millimeter geometric ground truth, complex microgravity procedures require continuous understanding of hand grasping and action dynamics. PARIKSHAK incorporates four deep learning neural architectures:

1. YOLOv8 Object Detection & Multi-Object Tracking (`YoLoDetector`)

- Uses Ultralytics YOLOv8 with ByteTrack for high-speed multi-entity localization.
- Maps detected object bounding boxes and track IDs into the procedure entity registry.

2. YOLO-Pose Crew Body Tracking (`YoLoPoseEstimator` & `YoLoHands`)

- Extracts 17 COCO body keypoints (wrists, elbows, shoulders, hips, knees, ankles).

- Projects foot and ankle coordinates into rack space to monitor crew anchorage in restraint foot loops.
- Kinetically derives 21 hand landmarks per arm from elbow-wrist vectors and palm geometry, with seamless fallback to MediaPipe BlazePalm.

3. Neural ContactMLP Grasp Classifier (`parikshak/perception/contact.py`)

- Evaluates a 12-dimensional kinematic interaction feature vector:

$$\mathbf{x}_{\text{contact}} = [d_{\text{wrist}}, d_{\text{min}}, d_{\text{mean}}, \text{aperture}, f_{\text{inside}}, \mathbb{I}_{\text{wrist_in}}, \text{IoU}, \text{diag}, v_{\text{close}}, v_{\text{obj}}, s_{\text{det}}, s_{\text{hand}}]$$
- Architecture: Dense(12 $\rightarrow$ 64, ReLU) $\rightarrow$ Dense(64 $\rightarrow$ 32, ReLU) $\rightarrow$ Dense(32 $\rightarrow$ 5, Softmax).
- Classifies 5 interaction states: none, reach, grasp, manipulate, release.
- Weights exported to `builds/contact_mlp.npz` (SHA-256 verified) with a pure NumPy forward execution engine.

4. Dilated Temporal Convolutional Network for Actions (`MotionTCN`, `parikshak/perception/motion.py`)

- Operates on a 20-timestep (2.0 s @ 10 Hz) sliding window of 16-channel kinematic vectors tracking hand velocities, grip aperture, contact probability, object velocity, and directional alignment.
- Architecture:
 - Input Conv1D(16 $\rightarrow$ 64, kernel=3) + ReLU
 - Residual Block 1: Conv1D(64 $\rightarrow$ 64, kernel=3, dilation=1) + ReLU + Identity Skip
 - Residual Block 2: Conv1D(64 $\rightarrow$ 64, kernel=3, dilation=2) + ReLU + Identity Skip
 - Residual Block 3: Conv1D(64 $\rightarrow$ 64, kernel=3, dilation=4) + ReLU + Identity Skip
 - Temporal Global Average Pooling (64,)
 - Classifier Head: Dense(64 $\rightarrow$ 32, ReLU) $\rightarrow$ Dense(32 $\rightarrow$ 7, Softmax)
- Classifies 7 action classes: idle, reach, insert, rotate_seal, press, agitate, withdraw.
- Weights exported to `builds/motion_tcn.npz` with zero-dependency pure NumPy forward inference.

6. Scientific experiment procedures (CSP-1 & CRX-2)

Experiment 1: CSP-1 (Colloidal Suspension Sample Processing)

- **File:** `procedures/csp1_colloid_sample_processing.yaml`
- **Steps:** 14 discrete operations.
- **Workflow:**
 1. S01: Restraint verification (crew anchored in foot restraints).
 2. S02 & S03 (Unordered group): Retrieve sample cartridge SC-A and vial A from storage locker.
 3. S04: Verify cartridge barcode label against camera.
 4. S05: Open glovebox door latch.
 5. S06: Insert sample cartridge into holder socket H1.
 6. S07: Align vial A to cartridge interface and rotate to lock seal.

7. S08: Close glovebox door latch (**Critical Step**).
8. S09: Press processing unit START push-button.
9. S10: Monitor 60-second processing cycle (**Safety Invariant**: latch must remain closed while centrifuge runs; **Fault Branch**: route to emergency shutdown if red indicator illuminates).
10. S11: Open glovebox door latch.
11. S12: Withdraw processed cartridge from socket H1.
12. S13: Insert cartridge into containment sample bag SB-01 and seal.
13. S14: Stow sealed sample bag into cold storage locker L1.

Experiment 2: CRX-2 (Colloid Resuspension)

- **File:** procedures/crx2_colloid_resuspension.yaml
- **Steps:** 8 discrete operations.
- **Workflow:**
 1. S01: Restraint verification.
 2. S02: Retrieve vial B from cold locker L1.
 3. S03: Confirm processing unit is in idle state.
 4. S04: Resuspend vial B by manual agitation (**Motion Count**: 10 agitation cycles; **Geometric Invariant**: vial must not approach within 12 cm of powered processing unit).
 5. S05: Place agitated vial B in settling tray T1.
 6. S06: Optical confirmation of colloid suspension uniformity.
 7. S07: Return vial B to cold locker L1 and latch door.
 8. S08: Crew verbal confirmation of run completion.

Zero Retraining Demonstration: CRX-2 was added to the repository purely as a YAML file. It reuses the same rack geometry, entity classes, and perception models without retraining or modifying code.

7. Software components & codebase organization

```
d:/projects/174ANTI/
├── parikshak/
│   ├── belief/
│   │   ├── frame.py
│   │   └── trace.py
│   ├── engine/
│   │   ├── accumulator.py
│   │   ├── alerts.py
│   │   ├── deviations.py
│   │   ├── hsmm.py
│   │   ├── log.py
│   │   ├── predicates.py
│   │   ├── procedure.py
│   │   └── truth.py
│   ├── pdl/
│   ├── perception/
│   │   ├── backends.py
│   └── MediaPipe)
│       ├── contact.py
│       └── motion.py
└── # Core on-board flight package (STRICTLY OFFLINE)
    # BeliefFrame contract and streaming trace I/O
    # Telemetry contract definitions
    # TraceWriter / TraceReader streaming engines
    # Procedure compliance reasoning engine
    # CUSUM continuous evidence accumulation
    # Multi-tier alert policy and persistence filtering
    # Typed deviation detector
    # Frontier step tracking & duration priors
    # Cryptographic SHA-256 hash-chained logger
    # 15 closed-vocabulary predicate evaluators
    # Unified ProcedureEngine facade
    # Three-valued interval logic implementation
    # Procedure Description Language parser & validator
    # Dual-tier perception stack
    # Model adapters (AprilTag, YOLO, YOLO-Pose,
```

	pipeline.py	# PerceptionPipeline belief assembly
	rackframe.py	# 6-DOF PnP rack pose estimator & AprilTag detector
	types.py	# Detections, Keypoints, HandLandmarks types
	io/	# Hardware interfaces
	camera.py	# OpenCV, synthetic, and GStreamer video sources
	clips.py	# Circular video buffer for ±10s alert capture
	downlink.py	# CFDP telemetry packet packager
	voice.py	# Synthesized speech and audio prompts
	gui/	# Desktop operator HUD (PySide6 window)
	eval/	# World-model simulator and perturbation harness
	live.py	# Live execution session runner
	run.py	# parikshak-run CLI entrypoint
	replay.py	# Deterministic trace replay engine
	demo/	# Ground-side browser demo application
	server.py	# FastAPI & Uvicorn application server
	guided.py	# Interactive step-by-step experiment simulator
	scenarios.py	# 14 recorded run scenario loaders
	procedures/	# Formal YAML experiment definitions
	csp1_colloid_sample_processing.yaml	
	crx2_colloid_resuspension.yaml	
	_template.yaml	# Template for authoring new procedures
	racks/	# Rack fiducial layouts and prop marker definitions
	schema/	# JSON schemas and predicate specifications
	builds/	# Neural weights and perception build manifests
	contact_mlp.npz	# Serialized ContactMLP neural network weights
	motion_tcn.npz	# Serialized MotionTCN temporal convnet weights
	parikshak-perception-1.2.0.json	
	traces/	# Golden runs and degraded evaluation corpora
	golden/	# 9 golden recorded traces for CSP-1
	golden_crx2/	# 7 golden recorded traces for CRX-2
	eval/	# 130 degraded evaluation traces for CSP-1
	eval_crx2/	# 70 degraded evaluation traces for CRX-2
	tools/	# Developer, evaluation, and documentation tools
	check.py	# All-in-one quality gate checker
	eval_report.py	# Quantitative evaluation and ROC sweep tool
	make_docs.py	# Generates HTML, PDF, and DOCX documentation
	make_demo_page.py	# Builds standalone offline HTML demo
	train_contact_mlp.py	# ContactMLP training pipeline
	train_motion_tcn.py	# MotionTCN training pipeline
	verify_log.py	# Hash-chain integrity verifier
	bench.py	# Real-time frame throughput benchmark
	soak.py	# 45-minute continuous endurance test
	deploy/jetson/	# NVIDIA Jetson Orin edge deployment package
	tests/	# 600+ automated test suite

8. Execution modes & command reference

8.1 Interactive Browser Demonstration

The fastest way to experience PARIKSHAK:

```
python -m demo
```

Opens an interactive dashboard in your browser (<http://localhost:8000>):

- **Perform an Experiment:** Interactive guided mode through CSP-1 or CRX-2 where you select crew actions (nominal, skip step, pick wrong object, open latch during run, block camera) and observe live engine decisions.
- **Watch a Recorded Run:** Replay 14 flight scenarios with synchronized timeline, rack visualization, audio prompts, and cryptographic log verification.

8.2 Standalone Offline Single-File Demo

To generate a self-contained HTML page embedding all 14 scenarios that runs offline without Python:

```
python tools/make_demo_page.py
# Output: runs/demo/parikshak_demo.html (opens in any browser by double-clicking)
```

8.3 Live Camera Execution

To run live on a connected USB webcam or space station camera feed:

```
# Using AprilTag printed fiducials (print from runs/tags/tags.pdf)
python -m parikshak.run --camera 0 --procedure
procedures/csp1_colloid_sample_processing.yaml --props racks/props_csp1.json

# Using live deep learning models (YOLOv8 + MotionTCN)
python -m parikshak.live --camera 0 --detector yolo --motion-classifier tcn
```

8.4 Deterministic Trace Replay

To replay a previously recorded experiment run through the reasoning engine:

```
# Replay in desktop operator HUD window with live checklist and rack overlay
python -m parikshak.replay traces/golden/skip_S08_latch.jsonl --window

# Replay in terminal with synthesized voice speech
python -m parikshak.replay traces/golden/skip_S08_latch.jsonl --speak
```

8.5 Cryptographic Log Verification

To audit a run log file and confirm that telemetry has not been modified post-mission:

```
python tools/verify_log.py runs/live/<run_id>.log.jsonl
```

8.6 Documentation Regeneration

To regenerate the formatted HTML, PDF, and DOCX documentation:

```
python tools/make_docs.py
```

9. Declarative experiment onboarding (zero-retraining workflow)

Adding a brand-new scientific experiment to PARIKSHAK requires four steps without modifying code:

1. **Author the Procedure YAML:** Copy `procedures/_template.yaml` and specify the experiment steps, rack zones, target entities, predicate truth conditions, duration priors, safety invariants, and alert policies.

2. **Schema & Semantic Validation:**

3. `python tools/validate_procedure.py procedures/my_experiment.yaml --strict`

Validates syntax against `procedure.schema.json`, verifies predicate argument validity, checks duration sums, and audits capability requirements.

4. **Verify Zero-Retraining Compatibility:**

5. `python tools/onboard.py procedures/my_experiment.yaml`

Checks all entity classes, zones, and predicates against the current perception build manifest (`builds/parikshak-perception-1.2.0.json`) to confirm that all required classes are supported without neural retraining.

6. **Synthesize Evaluation Corpus:** Use the world-model simulator to script nominal and failure runs, generate degraded corpora, and verify accuracy using `tools/eval_report.py`.

10. Evaluation methodology & robustness corpus

10.1 The Perturbation Corpus

Evaluation is performed across 200 synthetic runs (130 for CSP-1 representing 346.6 minutes; 70 for CRX-2 representing 58.7 minutes) subjected to four severity tiers:

1. **Clean:** Nominal ground truth with baseline sensor noise.
2. **Mild:** Minor Gaussian coordinate jitter ($\sigma = 3 \text{ mm}$), occasional detection confidence drops.
3. **Moderate:** Moderate coordinate noise ($\sigma = 8 \text{ mm}$), random frame dropouts (5%), partial fiducial obscuration.
4. **Harsh:** Severe coordinate noise ($\sigma = 15 \text{ mm}$), frame drops (15%), total camera blackout / body occlusion for up to 8 seconds.

10.2 Honest Scoring Protocol

To prevent optimistic bias:

- **True Deviations:** Evaluated strictly against injected errors (skips, wrong objects, hazards, rushed steps, stalled steps).
- **False Alarms:** An alarm counts as false if it is triggered during a nominal run, or if it concerns a step preceding an injected error (which the injected error could not have caused).
- **Cascade Disqualification:** Alarms on steps occurring *after* an injected error that legitimately cannot be verified due to invalid state are categorized as expected cascades and excluded from the false alarm penalty.

10.3 ROC Threshold Sweeps

Threshold sweeps (runs/roc_csp1.txt, runs/roc_crx2.txt) across completion bars (\$0.70 - 0.95\$) and deviation bars (\$0.70 - 0.95\$) confirmed the optimal operating point:

- **Completion Bar:** \$0.85\$
- **Deviation Bar:** \$0.85\$
- **Alert Persistence:** \$1.5\text{ seconds}\$

11. Quantitative verification results

11.1 Headline Verification Metrics

Measure	Target	CSP-1 (130 runs)	CRX-2 (70 runs)
Step Completion Accuracy	$\geq 95.0\%$	98.7 %	95.6 %
Sequence Deviation Recall	$\geq 90.0\%$	97.8 %	97.5 %
False Alarms per 45 Minutes	≤ 1.00	0.85	0.00
False Alarms on Permitted Reorders	0	0	0
False Alarms on Camera Occlusion	0	0	0
Alert Latency (Median / 95th Percentile)	$\leq 2.0\text{ s}$	1.6 / 1.6 s	1.6 / 1.8 s

- **Skips Caught:** 100 % on CSP-1; 95 % on CRX-2.
- **Hazards Caught:** 100 % on both experiments.
- **Wrong Objects Caught:** 100 % on both experiments.
- **Timing Violations Caught:** 90 % on CSP-1; 100 % on CRX-2.

11.2 Root Cause Analysis & False Alarm Reduction

Early prototypes registered 2.98 false alarms per 45 minutes on CSP-1. A systematic frame-by-frame root cause analysis identified three failure mechanisms, which were systematically resolved:

Root Cause	Physical Failure Mechanism	Engineering Solution	Alarms Eliminated
A. Premature Evidence Clearing	A momentary crew action occurring just before the reasoning engine caught up was erased when the previous step finalized.	Replaced blunt evidence clearing with bounded temporal lookback windows for occurred predicates.	4 false skips (S06)
B. Single-Frame Jitter Spikes	A single noisy frame was registered as "seen still undone", prematurely resetting the next step's start time and making it appear rushed.	Required contrary evidence to hold unbroken across an entire independent observation interval ($\Delta t = 0.5 \text{ s}$) before resetting duration clocks.	1 false "too fast"
C. Sensor Jitter Misread as Motion	Under severe noise, Gaussian position jitter on a stationary object exceeded the 3 cm movement threshold by statistical chance.	Implemented a two-standard-error noise gate: motion is affirmed only when displacement exceeds tolerance by $2 \times \text{SE}_{\text{jitter}}$.	5 false alarms (1 skip, 4 timing)

Result: False alarm rate dropped from 2.98 $\%$ to 1.92 $\%$ to **0.85 per 45 minutes**, meeting the stringent aerospace threshold while maintaining 98.7% step accuracy.

11.3 Residual Edge Cases & Failure Analysis

The remaining edge cases are documented in the failure reel (runs/reel/reel.mp4):

- CSP-1 Step S07 False Skips (4 occurrences under Harsh Noise):** Vial A is in active motion while being inserted; crew then closes the glovebox latch at 4.8 seconds—just 0.2 seconds below S07's 5.0-second physical prior.
- CSP-1 Step S12 Missed Rushes (2 occurrences under Harsh Noise):** A camera occlusion preceding the cartridge withdrawal causes the duration estimator to take a conservative stance, measuring elapsed time right at the boundary.
- CRX-2 Unverified States (7.3%):** Severe camera occlusion hides the vial immediately after placement in tray T1. The engine correctly reports "cannot verify" instead of guessing.

12. Real-Time performance & endurance benchmarks

12.1 Processing Throughput

Benchmarked on a standard laptop CPU (Intel Core 11th Gen, 8 threads) at 1280×720 resolution (runs/bench.json):

- **Full Perception & Reasoning Loop:** Median latency 34.6 ms (**28.9 FPS**); 95th percentile latency 74.8 ms (**13.4 FPS**). Exceeds the flight edge requirement of ≥ 10 FPS.
- **Procedure Reasoning Engine Alone:** Median latency **1.2 ms per frame**.

12.2 45-Minute Continuous Endurance Soak Test

Executed via `python tools/soak.py --minutes 45 --stream:`

- **Memory Stability:** Total memory growth was **7.4 MB per hour**, confirming strict bounding of all sliding temporal queues and history buffers.
- **Latency Stability:** Execution slowdown between the first and final 15 minutes was **1.35×** (well within the allowable 1.5× limit).
- **Streaming Trace Integrity:** The streaming `TraceWriter` output was confirmed byte-for-byte identical to memory-buffered traces.

13. Testing framework & CI/CD quality gates

Running the unified quality gate:

```
python tools/check.py
```

Executes all regression gates and must return **ALL GATES GREEN**:

1. **Procedure Structural Schema Gate:** Validates all YAML files against `procedure.schema.json`.
 2. **Capability & Build Gate:** Verifies procedure capability bounds against perception build manifests.
 3. **Zero-Retraining Gate:** Confirms that new procedures require no model weight updates.
 4. **Validator Negative Suite:** Tests 18 intentional syntax/semantic failure modes.
 5. **Comprehensive Test Suite:** Executes 600+ automated unit and integration tests (`tests/`).
 6. **Deep Learning Model Suite:** Verifies ContactMLP, MotionTCN, YOLOv8, and YOLO-Pose models (`tests/test_deep_learning_models.py`).
 7. **Golden Trace Replay Gate:** Replays all golden traces for CSP-1 and CRX-2, asserting exact match against expectation vectors.
 8. **Evaluation Metrics Gate:** Asserts that step accuracy $\geq 95\%$ and deviation recall $\geq 90\%$.
 9. **UI HUD & Speech Audio Gate:** Renders the operator display and exercises the text-to-speech audio path.
-

14. Project deliverables & artifact generation

Deliverable Artifact	File Location	Generation / Execution Command
Project Documentation (Markdown)	docs/PROJECT_DOCUMENTATION.md	Primary source document
Project Documentation (HTML/PDF/DOCX)	docs/PARIKSHAK_Project_Documentation.*	python tools/make_docs.py
Interactive Browser Demo	demo/	python -m demo
Offline Shareable Single-Page Demo	runs/demo/parikshak_demo.html	python tools/make_demo_page.py
SIH 2026 Presentation Deck	PARIKSHAK_SIH2026_Submission.pptx	python build_sih_pptx.py
Evaluation Metrics & ROC Curves	runs/eval.json, runs/eval_crx2.json	python tools/eval_report.py
Failure-Mode Video Reel	runs/reel/reel.mp4	python tools/make_reel.py docs/reel/segments.json
Downlink Telemetry Package Sample	runs/downlink_sample/	python tools/make_downlink_sample.py
Printable AprilTag Fiducial Sheets	runs/tags/tags.pdf	python tools/make_tag_sheets.py
Edge Hardware Bring-Up Package	deploy/jetson/	Documentation and deployment scripts

Deliverable Artifact	File Location	Generation / Execution Command
Neural Contact Classifier Weights	<code>builds/contact_mlp.npz</code>	<code>python tools/train_contact_mlp.py</code>
Temporal Action ConvNet Weights	<code>builds/motion_tcn.npz</code>	<code>python tools/train_motion_tcn.py</code>

15. Honest system boundaries

- Synthetic Evaluation Ground Truth:** The quantitative 200-run corpus was generated via the physical world-model simulator. While realistic noise, occlusions, and jitter were injected, real flight microgravity optical footage will exhibit fluid meniscus distortions, floating debris, and lighting shifts.
- Physical Webcam Lighting & Motion Blur:** Under low-cost webcams, extreme motion blur can degrade fiducial corner detection. In flight, high-shutter-speed global shutter industrial cameras (e.g., Basler / FLIR) resolve this issue.
- Hardware Power Measurement:** Real-time throughput was profiled on an Intel x86 laptop CPU (28.9 FPS). Power consumption ($\leq 25\text{ W}$) requires deployment onto the target NVIDIA Jetson Orin board with a hardware power meter.
- Speech Recognition Input:** Crew prompts are currently voiced by the system (pyttsx3 / Piper). Astronaut voice commands are received via typed/clicked inputs until an on-board Whisper/VOSK speech recognizer is integrated into avionics.

16. Flight certification roadmap (TRL progression)

[TRL 4: Component Validation in Lab] → Currently Achieved (600+ tests, 200 eval runs)



[TRL 5: High-Fidelity Mockup Testing]

- ├ Parabolic flight campaign / physical glovebox test rig
- ├ 60 recorded real-crew runs (30 nominal, 30 error injected)
- └ Microgravity dataset training (MicroG-4M, IndustReal)



[TRL 6: Prototype Demonstration on Space Hardware]

- ├ NVIDIA Jetson Orin deployment running TensorRT INT8 ($\leq 25\text{ W}$)
- ├ GStreamer RTSP hardware-accelerated video streaming
- └ Offline Piper neural text-to-speech & offline voice grammar engine



[TRL 7+: Flight Demonstration on BAS / Lunar Module]

17. Software licences & compliance audit

PARIKSHAK adheres to a strict open-source licensing policy suitable for defense and space agency procurement:

- **Permissive Runtimes:** Core dependencies (numpy, pyyaml, jsonschema, opencv-python, fastapi, uvicorn) are licensed under MIT, BSD-3-Clause, or Apache-2.0.
- **Desktop Window UI:** PySide6 is licensed under LGPL-3.0 and dynamically linked, permitting use without contaminating proprietary or government space codebases.
- **Copyright Isolation:** No AGPL-licensed libraries (such as vanilla Ultralytics commercial binaries) are linked into flight builds. All deep learning neural inference engines for ContactMLP and MotionTCN are implemented in clean-room pure NumPy.
- Full compliance statement: [docs/LICENCE_STATEMENT.md](#).

18. Technical glossary

Term	Technical Definition
BeliefFrame	The formalized telemetry data contract emitted by perception at every timestep, containing rack-frame object coordinates, bounding boxes, hand landmarks, contact states, and confidence intervals.
CUSUM	Cumulative Sum control chart; a sequential analysis technique used by PARIKSHAK to integrate positive support and contrary evidence continuously over time.
PDL	Procedure Description Language; the declarative YAML specification defining experiment steps, predicates, duration priors, and safety invariants.
Predicate	A discrete condition evaluating an entity's spatial, geometric, or state properties, resolving to a three-valued truth interval.
Three-Valued Truth	An interval $[lo, hi] \subseteq [0, 1]$ where unknown is explicitly distinct from false, preventing unverified sensor dropouts from triggering accusations.

Term	Technical Definition
Frontier	The active subset of procedure steps that the crew is permitted to be executing at the current timestep based on dependency ordering.
Duration Prior	A step's empirical minimum ($t_{\min}$), nominal (t_{nom}), and maximum ($t_{\max}$) allowable execution times.
Safety Invariant	A mandatory safety condition that must remain true continuously throughout the entire duration of a step.
Rack Frame	The canonical 3D Cartesian coordinate system fixed to the physical experiment rack by corner AprilTag fiducials.
Hash Chain	A sequential data structure where each record embeds the SHA-256 cryptographic digest of the prior entry, ensuring mathematical tamper evidence.
CFDP	CCSDS File Delivery Protocol; the international space standard file transfer protocol for space-to-ground downlink.